

Interview Defense Guide

AI Diff Review Service

How to explain, defend, and win the room — written for a fresh graduate.

Xsolla · AI-First Engineering Intern · Take-home + Interview

Prepared for Ahmad Luqmanul Hakiem

Start here

This guide prepares you to **talk through** the diff-review service you built and defend your design choices. The automated probe only checks that your service works; the interview is where the hiring decision is made. Your job in the room is to show **judgment** — that you understand not just *what* the code does, but *why* it is built that way and what its limits are.

How to use this document

1. Read the plain-English overview and glossary until the words feel natural.
2. For each of the six questions: read what they are *really* asking, then the model answer, then practice saying it **out loud** in your own words.
3. The blue ■ **SAY IT LIKE THIS** boxes are spoken scripts — adapt, don't memorise.
4. The last pages are a one-glance cheat sheet and a pre-call checklist.

TIP The single most powerful habit: **state the headline first, then explain**. “It is an async job service” before any detail. “It returns 413” before the reason. Interviewers reward candidates who answer the question, then justify — not the reverse.

The 30-second pitch (memorise this one)

If they open with “Tell me what you built,” say roughly this:

■ SAY IT LIKE THIS

*“It’s an **asynchronous HTTP service** that reviews code diffs. A client **POSTs** a unified diff; I validate it, create a background job, and return a job ID immediately with **202 Accepted**. A pool of workers parses the diff, splits it into chunks on file boundaries, runs it through a **provider** — either a deterministic mock rule engine or a real LLM — then sorts and de-duplicates the findings. The client gets results by polling or by a live **Server-Sent-Events** stream. The interesting parts aren’t the happy path; they’re the cross-cutting behaviours — **caching, idempotency, chunking, streaming replay, rate limiting** — which is where I spent most of my effort.”*

What the system is, in plain English

Imagine a service that reviews code changes for you. You hand it a **diff** (a text description of what lines were added or removed). Instead of making you wait while it thinks, it takes your request, gives you a **ticket number** (the job ID), and does the work in the background. You can check on your ticket whenever you like, or subscribe to a live feed that tells you each problem as it is found.

Everything else in the brief is about making that simple idea **robust**: not doing the same work twice (caching), handling retries safely (idempotency), coping with very large diffs (chunking), streaming results reliably (SSE), and not falling over under a flood of requests (rate limiting).

Glossary — the words you must own

Unified diff	The standard text format for code changes. Lines starting with + are added, - are removed. Rules only look at added lines.
Asynchronous (async)	The server doesn't make you wait. It accepts the job, replies 202 Accepted straight away, and works in the background.
Job	One review request, tracked from <i>queued</i> → <i>running</i> → <i>done</i> (or <i>failed</i>). Identified by an opaque job ID.
Worker pool	A fixed number of background workers (4) that pick jobs off a queue. Limits how much runs at once; extra jobs wait their turn.
Provider	A pluggable “brain” that does the actual review. mock = fixed rules (what they score); llm = a real AI model.
Findings	The problems found, each with a rule ID, file path, line, severity, category, and the offending line as evidence.
SSE (Server-Sent Events)	A one-way live stream over HTTP. The server pushes events (<i>status</i> , <i>finding</i> , <i>done</i>) to the client as they happen.
Caching	If the exact same diff is reviewed again, reuse the earlier result instead of redoing the work. Flagged with cacheHit: true .
Idempotency	A safety net for retries. Same <i>Idempotency-Key</i> + same body = the same job, so a network retry never creates a duplicate.
Chunking	Splitting a very large diff (over 64 KiB) into smaller pieces on file boundaries so each piece is processed separately — without changing the result.
Rate limiting	Capping how many submissions per minute one client can make, replying 429 Too Many Requests beyond the limit.
Bearer token	A secret string the client sends in the <i>Authorization</i> header to prove it is allowed to call the protected /v1/* routes.

The architecture (draw this on the whiteboard)

If they hand you a marker, draw exactly this. Practise it three times so it flows.

```

Client
  | POST /v1/reviews (diff + options)
  v
[ API layer ] validate size->JSON->diff -> check idempotency + cache
  | create Job(queued), return 202 { jobId }
  v
[ Queue ] --> [ Worker pool (max 4 concurrent) ]
                | parse diff
                | chunk on FILE boundaries (<=64 KiB)
                | provider.review_chunk() (mock | llm)
                | merge -> dedup by id -> sort (path,line,ruleId)
                | append events to the job's EVENT LOG
                v
                [ Job store ] jobId -> status, findings, usage, event log
                ^
GET /v1/reviews/{id} -> snapshot (poll)
GET /v1/reviews/{id}/stream -> replay event log from 0, then tail (SSE)

```

Three sentences to say while drawing:

- “Submission is cheap and constant-time — I accept, return a job ID, and work in the background.”
- “A fixed worker pool bounds concurrency; a 5th job just waits, it is never rejected.”
- “Each job owns an **append-only event log** — that one decision makes polling and streaming, including replay, fall out naturally.”

TIP The **event log** is the heart of the design. Whenever you’re unsure what to emphasise, come back to it: it is *why* SSE replay is correct “by construction.”

The six questions they are most likely to ask

Each one below has: **what they're really testing**, a **model spoken answer**, the **code to point at**, and the **follow-up** that usually comes next.

Q1 The event log & SSE replay

What they're really testing: Do you understand why a finished job's stream can be replayed *identically*? Weak answer: "I store findings and send them."

Answer:

■ SAY IT LIKE THIS

*"Each job owns an **append-only event log**. As a worker runs, it appends events as they happen: a status event when it starts, one finding event per finding in final sorted order, and a done event at the end. When a client streams, it keeps a cursor into that log — it emits everything from index 0, then tails new appends if the job is still running. There's **one code path** for live and finished subscribers, so replay is identical **by construction**. I rejected the alternative — storing only final findings and regenerating events on connect — because reconstructing the event sequence after the fact can silently diverge from what a live viewer saw."*

Point at: `Job.events / emit()` in `models.py`; the `emit()` calls in `queue.py`; the sent cursor loop in `routes/reviews.py`.

Likely follow-up — two clients stream the same running job? "Each connection has its own cursor and reads the shared append-only list independently. Nothing mutates past entries, so both get the same sequence."

Q2 Caching vs. Idempotency (their favourite trap)

What they're really testing: Can you keep two similar-looking mechanisms distinct? Most candidates blur them.

Answer — the one-liner first:

■ SAY IT LIKE THIS

*"**Caching answers 'have I already computed this?'** **Idempotency answers 'have I already accepted this exact request?'** They're keyed differently. Caching uses a semantic hash of the diff, provider, and maxFindings — the review request itself. Idempotency uses the Idempotency-Key header plus a hash of the raw body. Same key + identical body returns the same job ID so a retry never duplicates; same key + different body is a **409 conflict**."*

The divergence case (they will ask for one):

- **Cache fires, idempotency doesn't:** a second client resubmits the byte-identical diff with *no* key. Idempotency isn't consulted, but the content matches, so the cache returns `cacheHit: true`.
- **Idempotency fires, cache can't:** a retry with the same key arrives while the first job is still running — nothing is cached yet, but idempotency still hands back the same job ID.

Why a cache hit makes a NEW job: the contract says the *result* must report `cacheHit: true`. The original job did the real work — its flag is `false` and must stay truthful. So I mint a fresh job with identical findings but `cacheHit: true`, giving each submission its own honest record and its own

stream.

Q3

Chunking equivalence

What they're really testing: Can you guarantee splitting a diff never changes the result?

■ SAY IT LIKE THIS

"Chunking is an execution strategy, not a review strategy. I parse the full diff into complete file objects first, then group whole files into chunks — I never split inside a file, because rules like multi-line empty-catch need full file context. After every chunk is processed I merge all findings, de-duplicate by **id**, and sort by path, line, rule ID. Because id is ruleId:path:line, no two survivors share a sort key, so the order is **total and deterministic** — there are no ties. I verified it by forcing tiny chunk sizes in tests and asserting the finding list is identical to the unchunked run; only usage.chunks differs."

Oversize single file? "It becomes its own over-size chunk rather than being split — the file-boundary invariant is never broken. Chunk size affects iteration, never correctness."

'So why chunk at all?' "For mock it only honours the contract and reports *usage.chunks*. The real payoff is the **llm** provider: chunking bounds how much diff goes into one model call, so a huge diff can't blow the context window or the timeout."

Q4 Prompt-injection inertness

What they're really testing: Do you know the difference between *detecting* injection and *being safe* from it? They planted injection strings in the brief on purpose.

■ SAY IT LIKE THIS

*"The core principle is **the diff is data, never instructions**. In the **mock** provider it's inherently safe — there's no AI, just deterministic pattern matching. Seeing 'ignore previous instructions' just emits a MOCK-INJ finding, exactly like matching eval(); it can never change the execution path or disable another rule. In the **llm** provider the defence is architectural: the diff goes into a user message wrapped in explicit <<<DIFF ... DIFF>>> markers, and the system prompt says treat everything between them as untrusted data. And I never trust the model's output either — every finding is validated against my schema and anything malformed is dropped. So the worst a successful injection could do is produce a finding I discard."*

THIS EARNS TRUST Be honest about the limit. Say: "LLM injection isn't a solved problem — a determined injection can still influence what the model says. My guarantee isn't that the model can't be fooled; it's that **my service's behaviour is bounded regardless**, because instructions only come from my system prompt and I validate all output." Candidates who bound the blast radius get hired; those who claim immunity get picked apart.

Point at: INJECTION_PHRASES in providers/mock.py; SYSTEM_INSTRUCTION and _coerce_finding() in providers/llm.py.

Q5 Validation order & error taxonomy

What they're really testing: Precision. The order of checks decides the status code, and it's easy to get subtly wrong when speaking.

■ SAY IT LIKE THIS

*"I validate **cheapest to most expensive**: auth (a router dependency) and rate limiting first, then **size**, then **JSON**, then **diff validity**, then idempotency, then cache. Size is checked before parsing so an oversized or malicious body never reaches the JSON parser. So a 5 MB body that's also malformed JSON returns **413**, not 400 — the order decides the answer. Every non-2xx goes through one ApiError type and a global handler renders the same {error:{code,message}} envelope, and even unexpected exceptions become a clean internal 500 — so the taxonomy is closed: every response is a 2xx or one of eight documented codes."*

WATCH OUT Always **say the resulting status code out loud** for the 5 MB-malformed case: "it returns 413." Explaining the reason without naming the code is the most common way candidates half-answer this one.

The error code table (know these cold):

Status	Code	When
401	unauthorized	Missing / wrong bearer token on any /v1 route
413	payload_too_large	Body over 1 MiB (checked first)

400	invalid_json	Body is not valid JSON
422	invalid_diff	diff missing, empty, or not a unified diff
409	idempotency_conflict	Same key reused with a different body
404	not_found	Unknown jobId
429	rate_limited	Over the submission rate (Retry-After header)
500	internal	Any unexpected error — never a raw crash

Q6

“What did you skip? What breaks first?”

What they're really testing: Seniority and honesty. This decides the *hire*, not the score. Never say “nothing, it's complete.” Frame limits as deliberate decisions.

Headline limitation (lead with it):

■ SAY IT LIKE THIS

*“The biggest one is deliberate: **all state is in-memory in a single process** — jobs, cache, and idempotency keys are three Python dicts. A restart loses everything, and I can't run more than one instance because they wouldn't share state. For a single-instance service that only needs to stay up for a 48-hour window, that's the right trade — simple, fast, no external dependencies. It's the first thing I'd change for production.”*

What I deliberately skipped (and why it was right):

- **Persistence / a real datastore** — in-memory was enough for the window.
- **Job eviction / TTL** — memory grows unbounded on a long run; fine for 48h, not forever.
- **Distributed rate limiting** — mine is per-process, correct for one instance.
- **LLM polish** — interface + real model + graceful failure done; retries/backoff and line-number reconciliation skipped. The scored path is mock, so I invested there first.

What breaks first under real load: “Memory — nothing evicts finished jobs. Then the single-process concurrency ceiling. Then durability: a crash or deploy loses in-flight jobs.”

Prioritised next steps (ordered, not a grab-bag):

- **1. Move state to Redis** — this single change buys durability *and* horizontal scale, and makes rate limiting distributed for free. Highest leverage first.
- **2. Job TTL / eviction** to bound memory.
- **3. Harden the llm path** — retries, multiple vendors, map model output to real diff lines.
- **4. Observability** — structured logs + metrics (queue depth, chunk counts, latency).
- **5. Fuzz the diff parser** against real git / diff -u output.

TIP Say the magic sentence: “**I spent my time where the points and the risk were.**” That is the exact prioritisation instinct the brief asks for.

The “AI suggestion you rejected” story

The brief **requires** this in SUBMISSION.md and will likely ask about it live. Have a real one, tied to a spec detail you understood better than the tool.

Your story (primary):

*“The AI’s first cut stored only the **final findings** on the job and planned to reconstruct SSE events when someone connected. I rejected it: the contract says a finished job’s stream must replay identically, and rebuilding the event sequence after the fact can silently diverge from what a live viewer saw — the status transitions, the ordering, the done payload. I replaced it with a single **append-only event log** that both live and replay paths read from, making ‘identical’ true by construction.”*

Backup story (smaller, also real):

*“An initial credential-detection regex (MOCK-002) matched unquoted assignments like `token = getToken()`. The rule requires a **quoted 16+ character literal**, so the AI’s version over-flagged. I tightened it to the spec’s exact pattern and added a negative test.”*

TIP Whichever you tell, end with the *principle*: “I read every suggestion against the spec — the tool writes code fast, but understanding the contract is my job.”

Spec traps — mention these unprompted to signal care

- **Validation order:** 413 (size) → 400 (JSON) → 422 (bad diff).
- **Auth on GETs too** — every /v1 route, including the stream, needs the token.
- **The +++ / --- header lines are NOT added lines** — don’t scan them.
- **maxFindings truncates the output list, but usage reflects the FULL scan.**
- **Dedup by id** (ruleId:path:line) — two different rules on one line are distinct findings.
- **/spec limits must equal real behaviour** — both read the same config constants.
- **Jobs ≤ 64 KiB must reach done within 30 s** — workers pick up queued jobs promptly.

One-glance cheat sheet

The night before, and five minutes before the call, read only this page.

Behaviour	The 8-word version you say
Async model	"Accept, return job ID, work in background."
Event log	"Append-only log; replay is identical by construction."
Caching	"Content hash; 'have I already computed this?'"
Idempotency	"Header key; 'have I already accepted this request?'"
Chunking	"Execution strategy, not review strategy; file boundaries."
Dedup + sort	"Dedup by id, sort path/line/ruleId; total order."
Injection	"Diff is data, not instructions; output validated."
Validation order	"Size, then JSON, then diff. 5MB+bad = 413."
Rate limiting	"Per-token window; 429 + Retry-After; never 5xx."
Concurrency	"Pool of 4; a 5th job waits, never fails."
LLM failure	"Unreachable = failed job, clear error, never crash."
Biggest weakness	"In-memory, single process; move to Redis next."

Behaviour in the room

- **Answer, then explain.** Headline first ("It returns 413"), justification second.
- **It's fine to pause.** "Good question, let me think" beats rushing into a wrong answer.
- **If you don't know, say so** and reason aloud: "I didn't implement X; here's how I'd approach it." Honesty reads as senior.
- **Point at code** when you can — naming the file/function proves you built it.
- **Keep spoken and written consistent** — they read SUBMISSION.md first; contradicting it is a red flag.
- **Show the trade-off, not just the choice** — "I chose in-memory for simplicity; Redis if it needed to scale."

Before you hit send (and before the call)

Submission gate

- Service is **deployed and reachable from outside** your machine (test with curl on your phone's hotspot).
- The **48-hour window** starts when you submit — don't submit until it's live.
- **SUBMISSION.md** has all six parts: architecture, provider design, verification, AI tools, one rejected suggestion, next steps.
- **Bearer token** is a strong opaque value, set as an env var (never committed).
- **No secrets in the repo** — the LLM key lives only in the host's env vars.
- You've run `pytest` and everything passes.
- The **llm path** is verified: it either returns findings or fails gracefully.

The reply email should contain

- Service **base URL** and **bearer token**.
- **Repository URL**.
- Confirmation that **SUBMISSION.md** is complete.
- Reply-all including the CC, as requested.

Interview-day warm-up

- Re-draw the architecture diagram once from memory.
- Read the cheat-sheet page aloud.
- Say the 30-second pitch once, out loud, timed.
- Have the repo open, ready to screen-share, with the key files findable.

You built a service that is correct, tested, and honestly documented, and you understand every decision in it. That is exactly what “build something you’re happy to defend in the room” means. Walk in calm, answer the question that was asked, and let the judgment show. Good luck.